Merging Consecutive Layers of a Multilingual Language Model: Annealed Absorption and Gradual Layer Flattening on Qwen3

Sumit Kumar Jha
Roll No. 26210180
IIT Gandhinagar
sumitjha@iitgn.ac.in

Abstract

I try to make Qwen3-0.6B-Base shallower by merging pairs of neighbouring layers instead of deleting them. Between 2 and 8 of its 28 layers are removed (and 4 from Qwen3-1.7B-Base), and every model is tested in six languages (en, hi, fr, es, de, zh) plus two that are never used for calibration or training (ja, ru). I compare two methods of my own against plain dropping, a drop + distillation control trained exactly like the methods, and LaCo. The first, AAM, keeps whichever layer of a pair works better on its own, fades the other one out and distills. The second, PRISM-Shrink, starts from the exact parallel union of the two layers and shrinks it down to one layer while distilling. AAM gives the best accuracy: +2.3 points over the control at 4 layers on 0.6B ($p = 0.002$), +3.2 on 1.7B, and the gain is significant on all three seeds. PRISM-Shrink has the smallest perplexity increase once 4 or more layers are removed (+16.6 % against +24.2 % for the control, mean of 3 seeds), but it does not improve accuracy. The gains are modest, though: with 4 or more layers gone from the 0.6B model every method is close to random guessing, and even AAM keeps only 21 % of the original's above-chance skill. If the goal is memory, 8-bit quantization saves far more at almost no cost.

1 Introduction

A decoder-only transformer runs its L layers one after another, and each layer adds an update to the token's running state, the residual stream: $h_{i+1} = h_i + f_i(h_i)$. Qwen3-0.6B-Base has $L = 28$ such layers (Qwen Team, 2025). Every layer that is removed is one sequential step less at inference time. The simplest way to remove layers is to delete the ones whose input and output are already similar, measured by angular distance (Gromov et al., 2024; Men et al., 2024). But deleting a layer throws away whatever it wrote into the residual stream, and merging is a way to keep the work of both layers of a pair in one.

I evaluate in several languages because a shared layer has to work for all of them, and a merge that is harmless in one language may not be in another. I measure damage with perplexity, few-shot accuracy on five benchmarks and an in-context copying probe.

My contributions are a controlled multilingual comparison (matched control, paired bootstrap tests, three seeds, and a check that the original model is competent on each test cell), the AAM method together with the observation that the choice of kept layer is what drives accuracy, and PRISM-Shrink, a gradual and distilled version of layer flattening (Xu et al., 2026) trained on ICL-packed text.

2 Background: from the basics to this work

Text is split into tokens (Qwen3 has a vocabulary of 151,936), and an embedding table turns each token into a vector of size d, which is 1,024 for the 0.6B model. The model predicts $p(x_t \mid x_{<t})$ and is trained with the cross-entropy $-\frac{1}{T}\sum_t \log p(x_t \mid x_{<t})$. The exponential of this loss is the perplexity, and lower is better.

Figure 1 shows one transformer layer, which computes

$$N(x) = \gamma \odot x \,/\, \sqrt{\tfrac{1}{d}\textstyle\sum_j x_j^2 + \epsilon}, \tag{1}$$

$$h' = h + \mathrm{Attn}\big(N(h)\big), \tag{2}$$

$$h_{\text{next}} = h' + \mathrm{MLP}\big(N(h')\big), \tag{3}$$

$$\mathrm{Attn}(x) = \mathrm{softmax}\big(QK^\top/\sqrt{d_h} + M\big)\,V, \tag{4}$$

$$\mathrm{MLP}(x) = W_d\big(\mathrm{SiLU}(W_g x) \odot W_u x\big), \tag{5}$$

where N is RMSNorm, Q, K, V are linear maps of x, M is the causal mask, and W_g, W_u, W_d are the gate, up and down projections; grouped-query

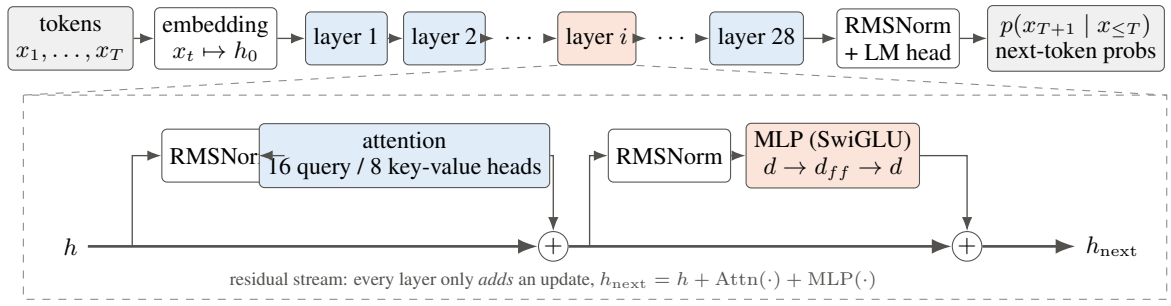


Figure 1: Top: a decoder-only language model turns tokens into vectors, refines them through 28 layers and predicts the next token. Bottom: inside one layer, attention and the MLP each read a normalised copy of the state and *add* their output to the residual stream.

attention shares each key-value head between 2 of the 16 query heads.

Since each layer only adds an update to h, the residual stream changes slowly. In the middle of the network the updates are small and neighbouring layers see almost the same input (cosine 0.96–0.97); merging neighbours relies on this.

In few-shot prompting the prompt contains solved examples, and a base model answers by continuing the pattern. This depends on *induction heads*, attention heads that find an earlier copy of the current token and predict the token that followed it (Olsson et al., 2022). Knowledge distillation (Hinton et al., 2015) trains a smaller or damaged student to match the teacher's output distribution (through a KL divergence) rather than only the correct token. Layer pruning deletes the layers whose input and output are most similar (Gromov et al., 2024; Men et al., 2024), and the deleted layer's contribution is gone unless some other part of the model learns it. For the statistics I use the bootstrap, which resamples the test questions to get a confidence interval, and a paired test, which compares two models on the same questions (Efron and Tibshirani, 1993).

3 Report

3.1 Feedback and novelties discussed with the mentor

This is what I have done since Presentation 2 (the repository history has the details). I first ran a small test of 15 merging algorithms, covering closed-form least squares, gradual distillation, search, the PRISM family and LaCo, with 100 questions per benchmark and language. I then moved to a large test with 200–250 questions per benchmark and language, more than 100k unique test tokens per language, per-question results so that bootstrap confidence intervals can be computed, and Japanese and Russian added as unseen languages. To separate what a method does from what the training alone does, I added a matched control (drop plus the same distillation). On top of that come a depth study (2, 4, 6 and 8 layers removed), a second model size (Qwen3-1.7B), a data ablation (kept layer $\times$ training text), a training-length ablation (400 vs 1,000 steps) and three seeds.

The new ideas I tried were PRISM (the exact parallel form of a layer pair), PRISM-Shrink (shrinking it gradually), AAM, ICL-packed training text, CoPlan pair selection, a language-specific residual (LSR), smeared-key merging (SKM) and a cross-term correction (XTC). SKM, XTC and LSR did not help, and I report them as negative results.

3.2 Baselines and state of the art

The closest earlier work to PRISM-Shrink is FlattenGPT (Xu et al., 2026). It merges two adjacent blocks into one wider block and then prunes it back to the original width in a single step, without training, and reports 87–96 % of zero-shot performance retained at about 20 % compression on 7B–70B English models. In my version the hand-over is gradual and distilled and starts from the exact original pair, the training text is ICL-packed, and the evaluation is multilingual with a matched control and several seeds.

The control is the baseline I care most about, because it gets the same training as the methods; any gain over it comes from the method and not from the extra training.

3.3 Datasets

Every language gets the same amount of calibration text, and every training step contains one se-

Term	Meaning in this report
Token, embedding	a piece of text; its vector of size d
Layer	one transformer block (attention + MLP); Qwen3-0.6B and 1.7B have 28
Residual stream	the running state h of a token; each layer adds its update to it
Attention head, KV group	one attention pattern; a key-value head with the 2 query heads that share it
MLP neuron	one of the d_{ff} hidden units of the feed-forward block
Perplexity	exp of the average next-token loss; how unsure the model is (lower is better)
Few-shot accuracy	share of correct answers when the prompt contains solved examples
Random guessing (chance)	25 % on 4-option tests, 33.3 % on XNLI, 0 % on MGSM; 21.7 % on the average
Skill kept	share of the original's above-chance accuracy that survives (0 = guessing, 100 = original)
Copying loss	loss on the second copy of a repeated random sequence; tests induction heads
Distillation (KD)	training a student to match a teacher's next-token distribution
Angular distance	angle between the state entering and leaving a pair; small = the pair changes little
Drop	delete one layer of each pair, no training
Control	drop + the same distillation as the methods (isolates the method's own effect)
AAM	keep the better layer of each pair, fade the other out, distill
PRISM-Shrink	run the pair as one wide parallel layer, shrink it gradually to one layer, distill
Bootstrap CI, paired p	range that contains the result 95 % of the time; chance of a difference this large if none existed

Table 1: Terms used in this report.

Method	Idea	Training	Code	Used here
Drop / ShortGPT (Men et al., 2024; Gromov et al., 2024)	delete the most similar layers	none	simple	yes (`drop`)
LaCo (Yang et al., 2024)	add other layers' weight differences to one layer	none	re-implemented	yes (small test)
Layer parallelism (Calvo González et al., 2025)	run two layers side by side	optional	public (GitHub)	idea only
LLM-Streamline (Chen et al., 2024)	train a small module to replace removed layers	yes	not checked	no
FuseGPT (Pei et al., 2024)	fold removed blocks into neighbours	yes	not checked	no
FlattenGPT (Xu et al., 2026)	one wide layer from two, pruned back in one step	optional	not released	closest prior work
Control (ours)	drop + the same distillation as the methods	yes	this repo	yes

Table 2: Baselines and related depth-compression methods.

Use	Data	Size per language
Calibration	Wiki-40B train (Guo et al., 2020)	16 × 256 tokens
Distillation	Wiki-40B train	300 × 256 tokens
Perplexity test	mC4 (Xue et al., 2021), Wiki-40B test	128 × 256 each
Knowledge	MMMLU (OpenAI, 2024; Hendrycks et al., 2021)	200 (5-shot)
Regional exams	INCLUDE (Romanou et al., 2024)	200 (5-shot)
Maths	MGSM (Shi et al., 2022)	250 (4-shot CoT)
Reading	Belebele (Bandarkar et al., 2023)	200 (3-shot)
Inference	XNLI (Conneau et al., 2018)	200 (3-shot)

Table 3: Data. Tests are never used for fitting or choosing anything. MGSM has no Hindi; INCLUDE has no English.

quence from each language. Calibration and training only use the Wiki-40B *train* split, and the pairs to merge are fixed by angular distance on calibration text before anything is tested, so nothing leaks from the test sets. One calibration sequence per language is held out to measure every fit. A test cell (benchmark × language) only counts if the original model is clearly above chance on it; the only flagged cell is German XNLI on 0.6B. Japanese and Russian are only ever evaluated and never used for anything else, so that damage outside the training languages shows up. All models get the same shots and prompts, and the shots never overlap with test items.

The main bottlenecks were GPU time (Kaggle T4s with a weekly quota) and GPU memory: the double-width layer in fp32 ran out of memory at 6 or more layers and on 1.7B. Hindi also needs 2–4× more tokens per item, and MMMLU has no development split, so I use its first shuffled rows as shots.

The data grew from the small test (100 questions per benchmark and language) to the large test, with 200–250 questions, 128 instead of 16 perplexity sequences, per-question results, and ja and ru added. That gives more than 100k unique test tokens per language (Table 13 in the appendix). I also built ICL-packed training text. Each sequence is $[a, b, a]$: a passage, a passage in another language, and the first passage again (768 tokens in total), so during training the model has to copy from its own context.

3.4 Experimental plan

3.4.1 Setting, training and robustness

I choose pairs by the angular distance $d(i) = \frac{1}{\pi} \arccos\cos(h_i, h_{i+2})$ averaged over tokens (Gromov et al., 2024), and never touch the first or last two layers. For Qwen3-0.6B this gives pairs 12-13 and 14-15 when 2 layers are removed and 10-11,

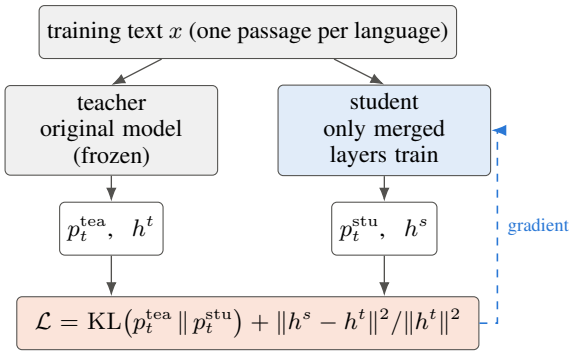


Figure 2: Knowledge distillation shared by the control, AAM and PRISM-Shrink: only the merged layers receive gradients.

	Qwen3-0.6B	Qwen3-1.7B
d / d_{ff}	1,024 / 3,072	2,048 / 6,144
n_q / n_{kv} / d_h	16 / 8 / 128	16 / 8 / 128
Layers L	28	28
P_{layer}	15,730,944	50,336,000
Embedding Vd	155,582,464	311,164,928
Total P	596,049,920	1,720,574,976
One layer	2.64 %	2.93 %
4 layers removed	533,126,144	1,519,230,976
Parameters cut	10.6 %	11.7 %

Table 4: Parameter counts (checked against the saved results). 2 / 6 / 8 layers removed from 0.6B: 5.3 / 15.8 / 21.1 %.

12-13, 14-15 and 16-17 for 4; removing 6 adds 7-8 and 24-25, and removing 8 adds 5-6 and 22-23.

All trained methods share one distillation setup (Hinton et al., 2015). The original model is a frozen teacher, and only the merged layers are trained, with the loss

$$\mathcal{L} = \mathcal{L}_{\mathrm{KL}} + \mathcal{L}_h, \tag{6}$$

$$\mathcal{L}_{\mathrm{KL}} = \tfrac{1}{T}\textstyle\sum_t \mathrm{KL}\big(p_t^{\text{tea}} \,\|\, p_t^{\text{stu}}\big), \tag{7}$$

$$\mathcal{L}_h = \|h^s - h^t\|^2/\|h^t\|^2, \tag{8}$$

where p_t is the next-token distribution at position t and h^s, h^t are the student's and teacher's hidden states at the output of each merged pair. The part being removed is faded out with

$$\alpha(s) = \tfrac{1}{2}\big(1 + \cos(\pi(s+1)/A)\big) \tag{9}$$

at step s, where $A = 0.7 \times 400 = 280$; for the last 120 of the 400 steps $\alpha = 0$ (Figure 13). At $\alpha = 1$ the student is exactly the original model, which I checked before training.

To keep the comparison fair, every method uses the same pairs, data, shots and training recipe. Beyond that there is the matched control, paired bootstrap tests on the same questions (Efron and Tibshirani, 1993), three seeds for the main 4-layer runs, the competence check, and two model sizes.

3.4.2 Models and approaches

Drop deletes the second layer of each pair, with no training. The **control** drops in the same way and then runs the shared distillation, so the first layer of each pair is kept and trained.

AAM keeps the layer with the lower held-out error when it is used alone. For pair 14-15, for example, the errors are 28.9 % and 26.6 %, so L15 is kept. The other layer becomes $h + \alpha(f(h) - h)$, is faded out during distillation, and is then removed.

PRISM starts from the observation that in the middle of the network $h + A(h) + B(h + A(h)) \approx h + A(h) + B(h)$, because the inputs of the two layers have cosine 0.96–0.97. The right-hand side is exactly one wider layer: the attention head groups and MLP neurons of A and B sit side by side, and each layer's RMSNorm gain is folded into its own input weights. This rewrite is exact; I checked it to $< 10^{-4}$ relative error.

PRISM-Shrink keeps 8 of the 16 key-value head groups and 3,072 of the 6,144 neurons, chosen by write energy. The output is $\alpha\, B(A(h)) + (1 - \alpha)\, U_\alpha(h)$, where U_α is the wide layer with every dropped part scaled by α. After distillation the kept parts are extracted as a normal layer.

LaCo sets $\theta = \theta_A + \sum_{j \neq A}(\theta_j - \theta_A)$ for every weight and needs no data.

The control and AAM run the same code and differ in a single switch, namely which layer of the pair is kept. The gap between AAM and the control therefore measures the layer choice, and the gap between PRISM-Shrink and the control measures merging against deleting. Data, pairs, loss, optimizer and number of steps are the same for all of them.

3.4.3 Parameters and architecture

One decoder layer with hidden size d, n_q query heads, n_{kv} key-value heads of size d_h and MLP width d_{ff} has

$$P_{\text{layer}} = 2\,d\,n_q d_h + 2\,d\,n_{kv} d_h + 3\,d\,d_{ff} + 2d + 2d_h, \tag{10}$$

and the model has $P = Vd + L\,P_{\text{layer}} + d$ (tied embeddings, vocabulary $V = 151{,}936$).

Every final model is an ordinary Qwen3 model with fewer layers of the same shape, so the architectures only differ during training. There, PRISM-Shrink holds a double-width layer for

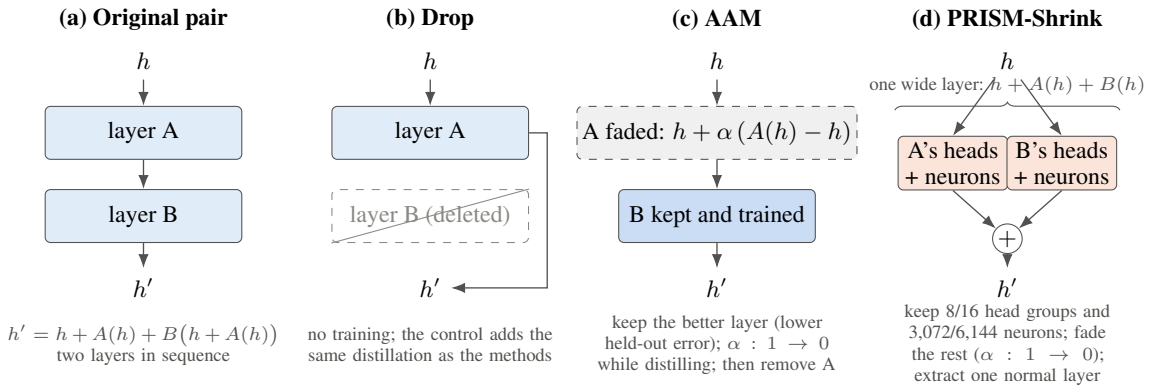


Figure 3: The four ways one pair of consecutive layers (A, B) is treated. Every final model has one standard layer per pair; AAM and PRISM-Shrink differ in what the remaining layer learns from.

Setting	Value
Optimizer	AdamW, lr 5×10^{-5}, no weight decay
Gradient clipping	1.0
Steps	400 (1,000 in the ablation)
Fade schedule	cosine over 280 steps, then 120 at 0
Batch per step	6×256 tokens (ICL: 6×768)
Loss	KL + 1.0 × hidden MSE
Precision	fp16 autocast (1.7B teacher in fp16)
Micro-batch	1,536 tokens (512 / 256 when memory-bound)
Calibration	16×256 tokens per language
Kept parts (Shrink)	8/16 head groups, 3,072/6,144 neurons
Seeds	0 (all runs); 1, 2 (main 4-layer runs)

Table 5: Training settings, identical for every trained method.

Kept layer	Text	Steps	Acc	PPL	Copy
first (control)	plain	400	23.3	+25.2	0.579
first (control)	ICL	400	23.5	+24.5	0.372
better (AAM)	plain	400	**25.8**	+22.8	0.337
better (AAM)	ICL	400	25.1	+21.6	**0.246**
better (AAM)	plain	1000	**26.6**	+20.6	0.513
union (Shrink)	plain	400	24.3	+17.5	0.218
union (Shrink)	ICL	400	23.5	**+16.5**	**0.212**
union (Shrink)	plain	1000	24.0	**+15.8**	0.339

Table 6: Ablations, Qwen3-0.6B, 4 layers removed. Acc = few-shot average accuracy (%), PPL = mC4 perplexity increase (%), Copy = copying loss (lower is better).

each pair (16 head groups, 6,144 neurons, plus a per-head q/k gain table of 5,632 parameters), while AAM and the control hold the two original layers with a fade between them.

3.4.4 Hyperparameters and ablations

I did not run a grid search. One Kaggle session per setting costs 1–3.5 GPU hours, so I fixed the recipe (learning rate, loss weight, fade length 0.7) on the small test and kept it identical for all methods. This keeps the comparison fair, but it also means no method was tuned for itself. In the ablations, the choice of kept layer moves accuracy by +1.6 to +2.5 points, while the training text moves it by at most 0.8. Training for 1,000 steps lowers perplexity by about 2 points and changes accuracy by at most +0.8 ($p = 0.05$, AAM), but copying gets worse (Table 6, Figure 7).

3.4.5 Metrics

For general language-modelling damage I use perplexity, $\exp\big(-\frac{1}{T}\sum_t \log p(x_t \mid x_{<t})\big)$, on mC4 and Wiki-40B, reported as the increase over the original model averaged over languages. For task ability I use few-shot accuracy on MMMLU, INCLUDE, MGSM, Belebele and XNLI, averaged over the five. Random guessing gives 25, 25, 0, 25 and 33.3 % on these, which is 21.7 % on the average. I also report the skill kept above chance, $\frac{1}{5}\sum_b \frac{a_b - c_b}{a_b^{\text{orig}} - c_b}$, and a copying probe in which 32 random 128-token sequences are shown twice and the NLL is measured on the repeat; this targets induction heads (Olsson et al., 2022). The remaining numbers are accuracy on the unseen languages, prefill and decode speed, and 95 % bootstrap confidence intervals with paired p-values.

3.4.6 System effort and demo scenario

The large-test queue logged 21.0 session-hours of Kaggle GPU time over 13 paired sessions, three of which failed and had to be re-run (Table 14 in the appendix). Earlier runs are not in this log. Each session uses two T4 GPUs, with one notebook per GPU.

Demo for the viva. A full 0.6B run takes 1–1.5 hours, so the demo has two parts. Live: run the CPU smoke-test version of a notebook (`tools/make_notebooks.py` builds it with tiny settings: 64-token sequences, a few training and

Resource	Used
GPU	Kaggle, 2 × NVIDIA T4 (15 GB each)
CPU / RAM	Kaggle T4 session: 4 cores, 29 GB
Disk	Kaggle working directory, 20 GB (results < 1 GB)
Orchestration	local laptop (Kaggle API)
GPU time	21.0 session-hours logged

Table 7: Computational resources.

test items), which goes through every stage (pair selection, AAM or PRISM-Shrink build, distillation, perplexity, benchmarks, copying probe) in minutes and without a GPU. Then load one compressed model and generate text from a prompt next to the original model. Saved: open the logged Kaggle runs and rebuild the tables and figures of this report from their JSON results with `tools/analyze.py` and `make_figures.py`.

3.5 Project management

Most of the new ideas came out of analysis or earlier failures. A layer analysis (angular change per layer, write sizes, update similarity, attention-head roles; Figure 14) showed that the middle layers barely change the state and read nearly the same input, and that is where the parallel form (PRISM) came from. Closed-form fits did not hold up on few-shot accuracy, which pushed me towards gradual distillation (AAM, PRISM-Shrink). The copying probe and the head analysis led to ICL packing.

4 Experiment

4.1 Implementation

The code is at `https://github.com/sumitjhaa/gpt2-layermerging` (branch `assignment3-top3`, folder `Assignment3/Sumit`). The pipeline (Figure 4) runs in five steps.

1. Download and cache Wiki-40B, mC4 and the five benchmarks, and build the calibration, training and test splits. A reference run shares these with all other notebooks.
2. Analyse the layers and choose the pairs by angular distance.
3. Build the method (`keep_one_build` for AAM and the control, `shrink_build` for PRISM-Shrink) and distill it.
4. Evaluate perplexity, the five benchmarks, the copying probe and speed, and write one JSON row per model that holds every question's result.
5. Generate one notebook per experiment (`tools/make_notebooks.py`), run them on Kaggle (`tools/kaggle_queue.py`), file the runs (`tools/organize_runs.py`), and rebuild all tables and figures (`tools/analyze.py`, `make_metrics.py`, `make_figures.py`).

4.2 Results and findings

With plain dropping, perplexity rises by 14 % at 2 layers and by 689 % at 8, and accuracy is already at chance from 4 layers on (Table 8, Figure 5). Distillation repairs much of this. At 4 layers the control brings the perplexity increase down from +84.7 % to +24.5 %, but its accuracy goes up by only 1.0 point ($p = 0.05$); on 1.7B the gain is larger, +6.8.

AAM beats the control by +4.8, +2.3, +1.5 and +1.8 points at 2, 4, 6 and 8 layers (all $p \leq 0.018$) and by +3.2 on 1.7B (Figure 9). Since AAM and the control differ only in which layer is kept, this gain comes from the layer choice; with the training text held fixed it is worth +1.6 to +2.5 points (Table 6). Over three seeds AAM gets 25.8±0.3 (Table 10, Figure 12).

PRISM-Shrink behaves differently. From 4 layers on, its perplexity is lower than both the control's and AAM's in all 6 languages, on mC4 and on Wikipedia, at every depth and on both models (Figure 11), and its copying loss is the lowest as well. Its accuracy, however, is never significantly different from the control's (-1.0 to $+1.0$, $p \geq 0.07$), so merging helps language modelling but not the benchmarks (Figure 10).

Model size matters a lot here: at 4 layers removed AAM keeps 41 % of the above-chance skill on 1.7B and only 21 % on 0.6B (Table 9, Figure 6). Maths goes first. MGSM falls from 41 % to about 2 % for every method at 4 layers (Table 11), and on Belebele drop and the control even end up below chance (about 20 %).

For saving memory, quantization is the better tool. 8-bit quantization halves memory with almost no loss on Qwen3, and 4-bit loses about 5 MMLU points on Qwen3-8B (Zheng et al., 2025), whereas removing 4 layers here saves 11 % and costs 23 points. In my view layer removal only makes sense for speed (roughly 10–20 % faster at 4 layers in my runs), and it can be combined with quantization.

4.3 Deployment

Every compressed model is a standard Qwen3 model with fewer layers, so it can be saved with `save_pretrained` and loaded with Hugging Face

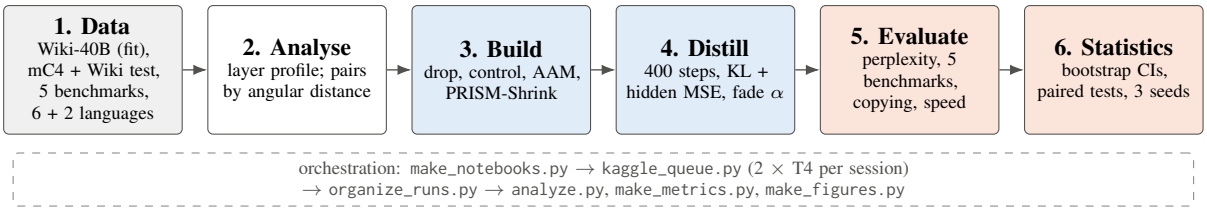


Figure 4: The end-to-end pipeline, from data to statistics, and the scripts that run it on Kaggle.

Removed	Method	Acc (%)	vs drop [95% CI]	Skill kept	mC4 PPL	Copy
0	original	48.9	–	100 %	–	0.125
2	drop	28.7	–	32 %	+14.2 %	0.320
2	control	29.3	+0.6 [−0.3, +1.5]	35 %	+7.9 %	0.172
2	AAM	**34.1**	**+5.4** [+3.9, +6.8]	**53 %**	**+6.0 %**	0.203
2	PRISM-Shrink + ICL	28.2	−0.4 [−1.5, +0.6]	31 %	+6.2 %	**0.131**
4	drop	22.6	–	7 %	+84.7 %	2.220
4	control	23.5	+1.0 [−0.0, +1.9]	13 %	+24.5 %	0.372
4	AAM	**25.8**	**+3.3** [+1.9, +4.6]	**21 %**	+22.8 %	0.337
4	PRISM-Shrink + ICL	23.5	+0.9 [−0.3, +2.3]	11 %	**+16.5 %**	**0.212**
6	drop	21.5	–	1 %	+236.6 %	4.292
6	control	22.3	+0.8 [−0.3, +2.0]	4 %	+59.2 %	0.506
6	AAM	**23.8**	**+2.3** [+1.0, +3.7]	**12 %**	+54.7 %	0.519
6	PRISM-Shrink + ICL	23.1	+1.6 [+0.4, +2.8]	7 %	**+39.4 %**	**0.340**
8	drop	20.9	–	−2 %	+689.4 %	10.112
8	control	21.6	+0.7 [−0.4, +1.9]	1 %	+103.8 %	1.091
8	AAM	**23.4**	**+2.5** [+1.1, +4.0]	**10 %**	+103.1 %	0.766
8	PRISM-Shrink + ICL	22.3	+1.4 [+0.3, +2.5]	4 %	**+69.0 %**	**0.651**

Table 8: Qwen3-0.6B-Base. Acc = few-shot average accuracy (random guessing 21.7 %); vs drop = paired difference on the same questions; skill kept = share of the original's above-chance accuracy; PPL = mean mC4 perplexity increase; Copy = copying loss.

Method	Acc	vs drop	PPL	Copy
original	58.9	–	–	0.106
drop	24.1	–	+70.5	2.831
control	30.9	+6.8	+32.1	1.150
AAM	**34.1**	**+10.0**	+29.4	0.450
PRISM-Shrink	31.5	+7.4	**+22.3**	**0.257**

Table 9: Qwen3-1.7B-Base, 4 layers removed (all methods trained on plain text). All differences vs drop $p < 0.001$.

Method	Acc (seeds 0/1/2)	Mean	vs control
AAM	25.8 / 25.4 / 26.1	25.8±0.3	+1.8 to +2.6*
PRISM-Shrink	23.5 / 23.7 / 23.5	23.6±0.1	−0.0 to +0.1
control	23.5 / 23.6 / 23.5	23.6±0.1	–

Table 10: Three seeds, Qwen3-0.6B, 4 layers removed. *significant on every seed ($p \leq 0.009$). Mean mC4 perplexity increase: AAM +23.0 %, PRISM-Shrink +16.6 %, control +24.2 %.

`transformers` without any custom code. I did not save the trained models in these runs; the notebooks rebuild them. The research pipeline itself is already runnable: one Kaggle notebook per experiment rebuilds the model, evaluates it and writes out the results (Section 4.1), and a 0.6B run takes about 1–1.5 hours on one T4. For inference the weights need about 1.2 GB (0.6B, fp16) or 3.4 GB (1.7B, fp16) of GPU memory. CPU inference works but is slow.

I chose an executable research pipeline over a web interface or API. The output of this project is a comparison of methods, not a product, and every number has to be reproducible from code; a free Kaggle GPU also cannot host a permanent service. End to end, the input is a model name, the layer count to remove and the method; the pipeline picks the pairs, builds and distills the shortened model, evaluates it, and outputs one JSON row of results plus the tables and figures. The same shortened model can be prompted directly with `generate` to show its text output. Production deployment is not the goal: at 4 or more layers removed the 0.6B model is close to random guessing, so it would not be useful to serve.

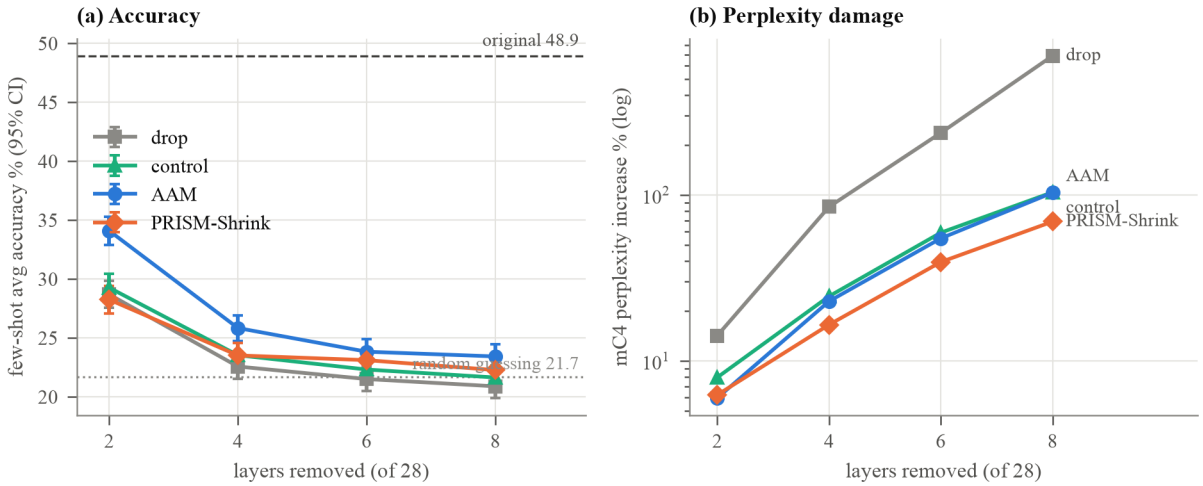


Figure 5: Accuracy (95% CI) and perplexity damage versus layers removed, Qwen3-0.6B.

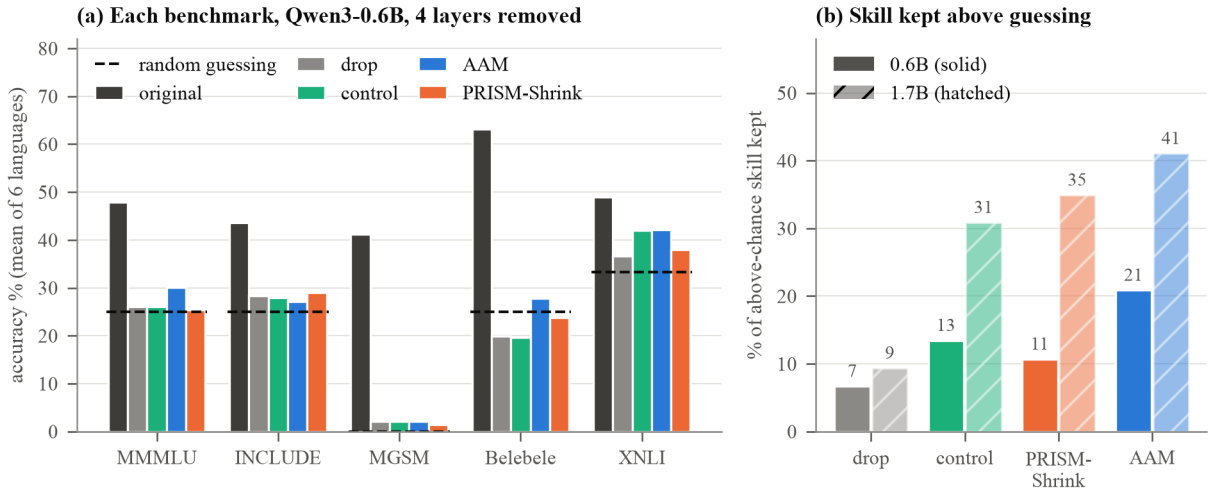


Figure 6: Left: each benchmark at 4 layers removed against random guessing (dashed). Right: share of the original's above-chance skill kept, 0.6B and 1.7B.

Limitations

With 4 or more layers removed from 0.6B, every method is close to random guessing and maths is lost entirely; this is the main weakness of the work. PRISM-Shrink gives no accuracy gain, and its core idea has already been published (Xu et al., 2026). Only the main 0.6B 4-layer runs have three seeds; every other setting is a single run. In the main 0.6B comparison AAM was trained on plain text while the control and PRISM-Shrink were trained on ICL-packed text. The text ablation puts this effect at 0.8 points at most, but the comparison is still not perfectly matched.

The study covers one model family, two small sizes, six languages and 200–250 questions per cell. Apart from MGSM there are no instruction-following or generation-quality tests, and the hyperparameters were not tuned per method. The quantization comparison comes from the literature rather than from my own test, and the speed numbers are noisy because the GPUs were shared. Finally, some earlier experiments survive only as numbers, since their code and raw results were never committed (see `docs/limitations_and_issues.md`).

Team Contribution

Member	Contribution	Score
Sumit Kumar Jha	data pipeline, methods, implementation, experiments, analysis, report	100 %

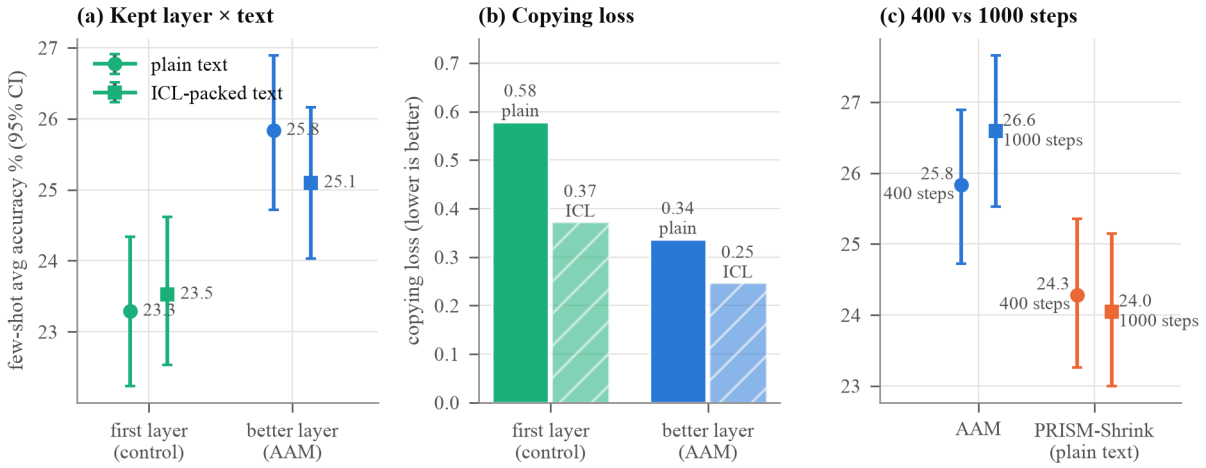


Figure 7: Ablations at 4 layers removed: kept layer × training text, copying loss, and training length.

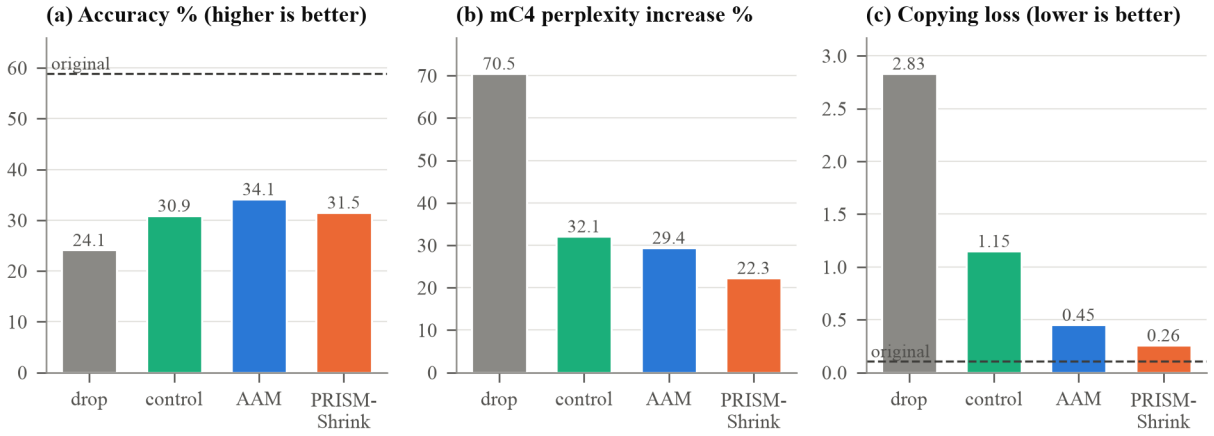


Figure 8: Qwen3-1.7B-Base, 4 layers removed.

References

Lucas Bandarkar et al. 2023. The Belebele benchmark: a parallel reading comprehension dataset in 122 language variants. *arXiv preprint arXiv:2308.16884*.

Ramón Calvo González, Daniele Paliotta, Matteo Pagliardini, Martin Jaggi, and François Fleuret. 2025. Leveraging the true depth of LLMs. *arXiv preprint arXiv:2502.02790*.

Xiaodong Chen et al. 2024. Streamlining redundant layers to compress large language models. *arXiv preprint arXiv:2403.19135*.

Alexis Conneau, Ruty Rinott, Guillaume Lample, Adina Williams, Samuel Bowman, Holger Schwenk, and Veselin Stoyanov. 2018. XNLI: Evaluating cross-lingual sentence representations. In *Proceedings of EMNLP*.

Bradley Efron and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman and Hall/CRC.

Andrey Gromov, Kushal Tirumala, Hassan Shapourian, Paolo Glorioso, and Daniel A. Roberts. 2024. The unreasonable ineffectiveness of the deeper layers. *arXiv preprint arXiv:2403.17887*.

Mandy Guo, Zihang Dai, Denny Vrandečić, and Rami Al-Rfou. 2020. Wiki-40B: Multilingual language model dataset. In *Proceedings of LREC*.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring massive multitask language understanding. In *International Conference on Learning Representations*.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.

Xin Men et al. 2024. ShortGPT: Layers in large language models are more redundant than you expect. *arXiv preprint arXiv:2403.03853*.

Catherine Olsson et al. 2022. In-context learning and induction heads. *arXiv preprint arXiv:2209.11895*.

OpenAI. 2024. MMMLU: Multilingual massive multitask language understanding. https://huggingface.co/datasets/openai/MMMLU.

Zehua Pei et al. 2024. FuseGPT: Learnable layers fusion of generative pre-trained transformers. *arXiv preprint arXiv:2411.14507*.

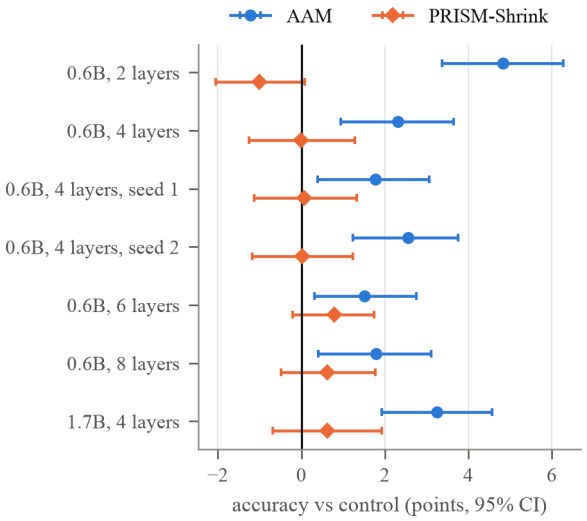


Figure 9: Accuracy difference to the matched control on the same questions (paired bootstrap, 95% CI). AAM is right of zero in every setting; PRISM-Shrink overlaps zero in every setting.

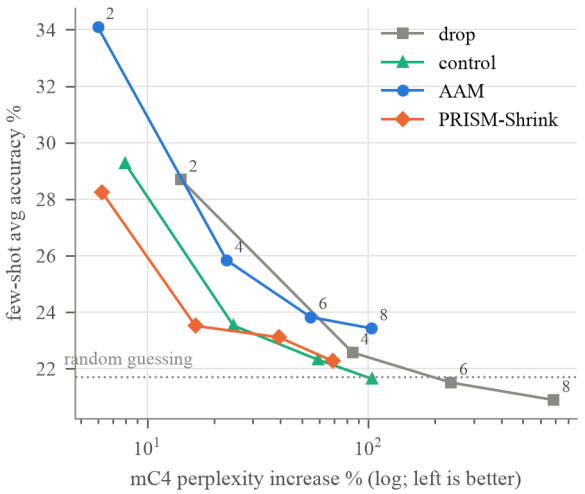


Figure 10: Accuracy against perplexity damage, Qwen3-0.6B; numbers are layers removed. AAM is highest, PRISM-Shrink is furthest left.

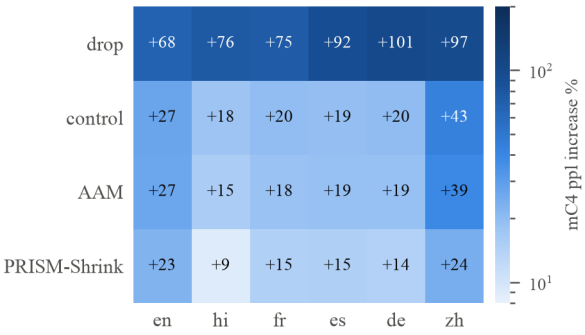


Figure 11: mC4 perplexity increase (%) per language, Qwen3-0.6B, 4 layers removed. Chinese is hurt most; PRISM-Shrink is lowest in every language.

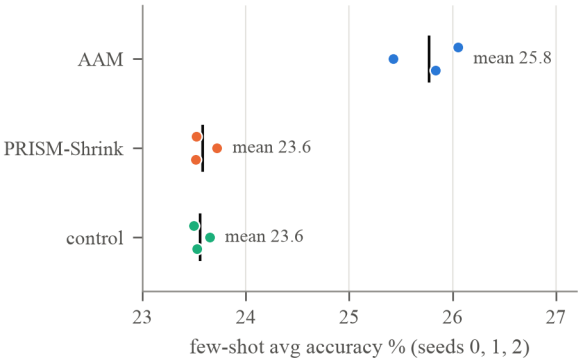


Figure 12: Three training seeds, Qwen3-0.6B, 4 layers removed; bars mark the means.

Qwen Team. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Angelika Romanou et al. 2024. INCLUDE: Evaluating multilingual language understanding with regional knowledge. *arXiv preprint arXiv:2411.19799*.

Freda Shi et al. 2022. Language models are multilingual chain-of-thought reasoners. *arXiv preprint arXiv:2210.03057*.

Ruihan Xu, Qingpei Guo, Yao Zhu, Xiangyang Ji, Ming Yang, and Shiliang Zhang. 2026. FlattenGPT: Depth compression for transformer with layer flattening. *arXiv preprint arXiv:2602.08858*.

Linting Xue et al. 2021. mT5: A massively multilingual pre-trained text-to-text transformer. In *Proceedings of NAACL*.

Yifei Yang, Zouying Cao, and Hai Zhao. 2024. LaCo: Large language model pruning via layer collapse. *arXiv preprint arXiv:2402.11187*.

Xingyu Zheng et al. 2025. An empirical study of Qwen3 quantization. *arXiv preprint arXiv:2505.02214*.

A Additional results

Method	MMMLU	INCL.	MGSM	Bele.	XNLI
chance	25	25	0	25	33.3
original	47.9	43.6	41.2	63.1	48.9
drop	26.0	28.3	2.2	19.8	36.6
control	26.1	27.9	2.1	19.7	41.9
AAM	30.1	27.1	2.1	27.8	42.1
PRISM-Shrink	25.4	29.0	1.4	23.8	38.0

Table 11: Accuracy (%) per benchmark, Qwen3-0.6B, 4 layers removed, mean of the six languages.

Benchmark	ours 0.6B	official	ours 1.7B	official
MMLU (en)	54	52.8	59	62.6
MMMLU	47.9	50.2	50.9	63.3
INCLUDE	43.6	34.3	54.6	45.6
MGSM	41.2	31.0	58.8	50.7

Table 12: Original models on this test vs the Qwen3 report (Qwen Team, 2025), Table 8. Differences come from the languages (6 here), question subsets, prompts and MGSM shots (4 vs 8).

Language	Unique test tokens	Perplexity tokens
en	139,093	65,536
hi	330,752	65,536
fr	186,368	65,536
es	181,921	65,536
de	182,852	65,536
zh	156,288	65,536
ja (unseen)	197,929	65,536
ru (unseen)	168,363	65,536

Table 13: Size of the large test per language (Qwen3-0.6B tokenizer).

Session	Outcome	Hours
reference (0.6B)	complete	3.03
data ablation	complete	1.31
deep methods	error (Shrink OOM)	1.94
reference (1.7B)	error (dtype)	1.73
deep control + Shrink plain	complete	3.51
reference (1.7B), re-run	complete	0.77
methods (1.7B)	error (Shrink OOM)	1.12
Shrink deep 6 / 8	complete	1.47
Shrink + control (1.7B)	complete	1.09
1000 steps	complete	1.27
seeds AAM	complete	1.05
seeds Shrink	complete	1.40
seeds control	complete	1.34
Total		21.0

Table 14: Logged Kaggle GPU sessions of the large-test queue (two T4 GPUs each). Failed sessions were fixed and re-run; the AAM halves of the two out-of-memory sessions are used.

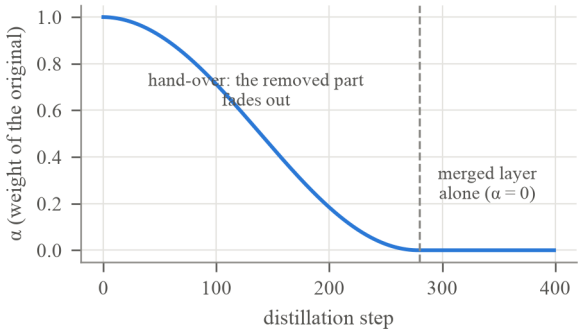


Figure 13: The fade schedule α over training steps.

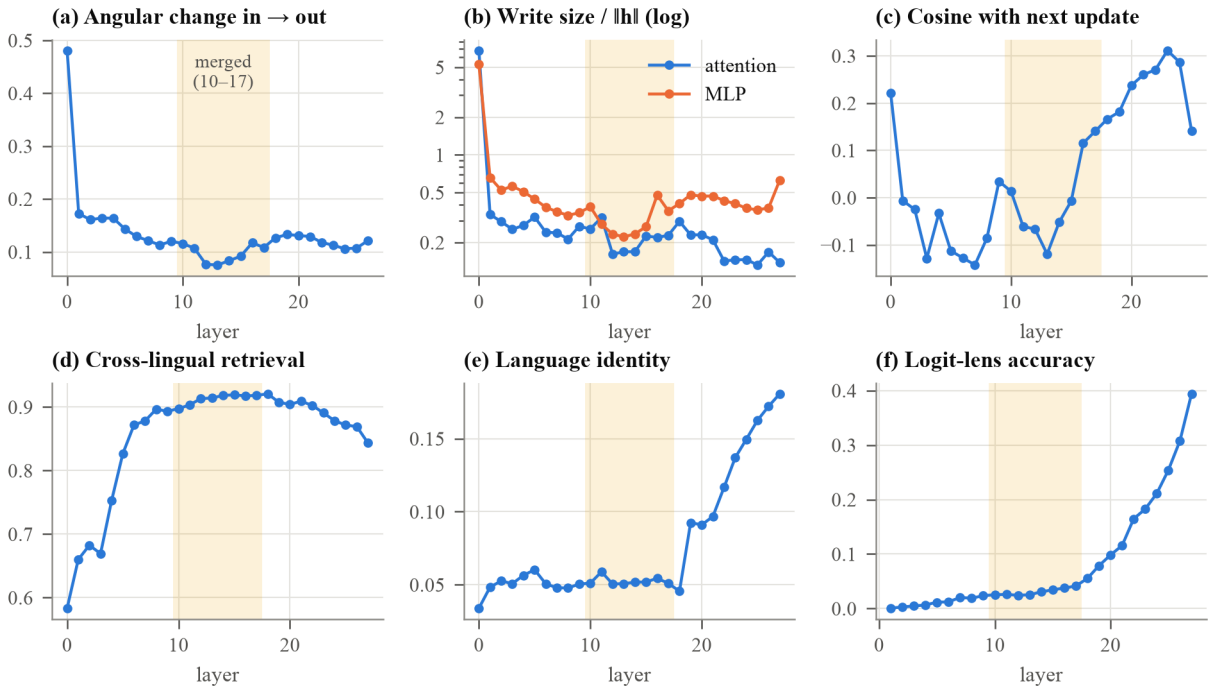


Figure 14: Layer analysis of Qwen3-0.6B (shaded: the layers merged at 4 layers removed). The middle layers change the state least (a) and write least (b); language identity and next-token readiness grow only in the last third (e, f).